

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. To learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

The complete implementation of the Single Layer Perceptron, including data preprocessing, exploratory data analysis, model implementation, training, evaluation, visualization, and additional tasks, has been uploaded to GitHub.

GitHub Repository:

<https://github.com/KPR-24110081/Deep-Learning-Lab/tree/main/Ex-1>

The repository contains:

- Jupyter Notebook (Ex_1.ipynb)

- Banknote Authentication Dataset
- Generated Plots
- Experiment Report
- README describing the implementation

7. Experimental Results

7.1 Dataset Overview

The Banknote Authentication dataset consists of 1372 instances with four numerical input features and one binary target variable. The dataset contains no missing values and is suitable for binary classification using a Single Layer Perceptron.

Dataset Preview

Table 1: First Five Samples of the Dataset

Variance	Skewness	Curtosis	Entropy	Class
3.6216	8.6661	-2.8073	-0.4469	0
4.5459	8.1674	-2.4586	-1.4621	0
3.8660	-2.6383	1.9242	0.1065	0
3.4566	9.5228	-4.0112	-3.5944	0
0.3292	-4.4552	4.5718	-0.9888	0

Dataset Dimensions

Table 2: Dataset Dimensions

Property	Value
Number of Samples	1372
Number of Features	4
Target Classes	2

Missing Values

Table 3: Missing Value Analysis

Feature	Missing Values
Variance	0
Skewness	0
Curtosis	0
Entropy	0
Class	0

Descriptive Statistics

	variance	skewness	curtosis	entropy	class
count	1372.0000	1372.0000	1372.0000	1372.0000	1372.0000
mean	0.4337	1.9224	1.3976	-1.1917	0.4446
std	2.8428	5.8690	4.3100	2.1010	0.4971
min	-7.0421	-13.7731	-5.2861	-8.5482	0.0000
25%	-1.7730	-1.7082	-1.5750	-2.4135	0.0000
50%	0.4962	2.3197	0.6166	-0.5867	0.0000
75%	2.8215	6.8146	3.1792	0.3948	1.0000
max	6.8248	12.9516	17.9274	2.4495	1.0000

7.2 Data Preprocessing

The numerical features were normalized using Min-Max Scaling to transform all feature values into the range $[0,1]$. The dataset was subsequently divided into training and testing sets using an 80:20 ratio while maintaining the class distribution.

7.3 Model Training

The Single Layer Perceptron was initialized with zero weights and bias. The model was trained using the Perceptron Learning Rule for 100 epochs with a learning rate of 0.01. During training, the number of misclassified samples gradually decreased, indicating that the model successfully learned the underlying decision boundary.

7.4 Model Evaluation

The trained perceptron was evaluated using the testing dataset. The performance was measured using Accuracy, Precision, Recall, F1-score, and the Confusion Matrix. The obtained metrics indicate that the perceptron effectively classified authentic and forged banknotes.

8. Generated Plots with Interpretation

8.1 Histogram of Features

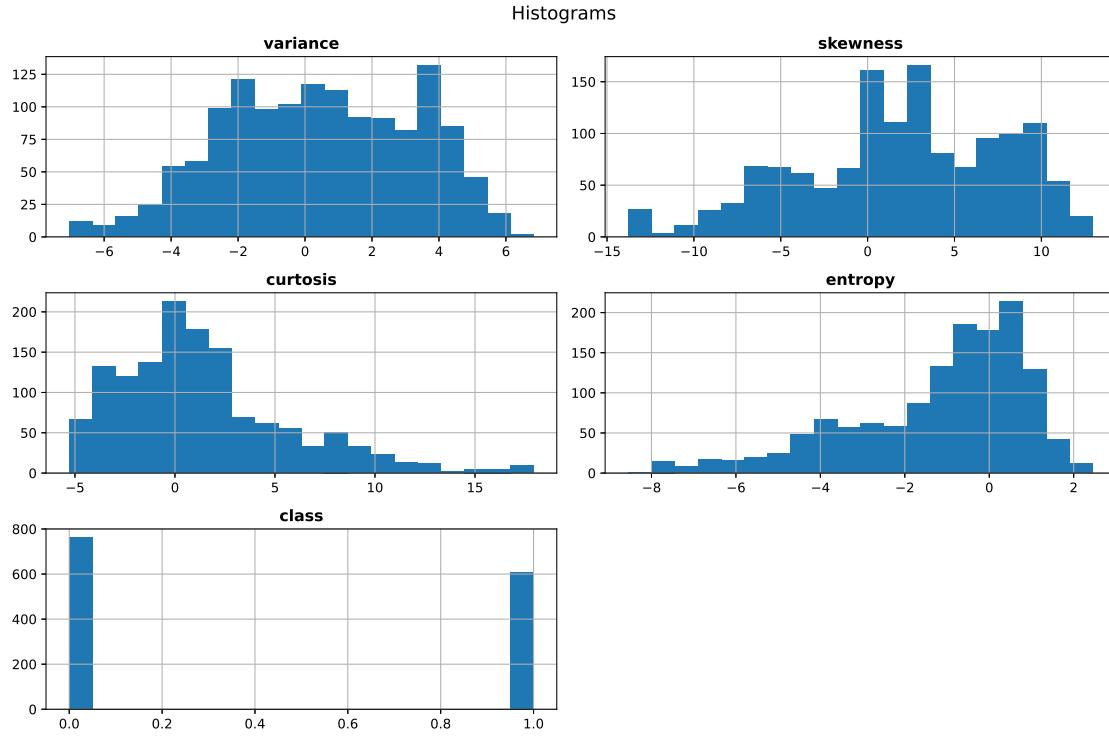


Figure 1: Histogram of the Banknote Authentication Dataset Features

Interpretation

The histograms illustrate the distribution of the four numerical features in the dataset. The features exhibit different ranges, spreads, and levels of skewness, indicating that feature normalization is necessary before training the perceptron.

8.2 Correlation Heatmap

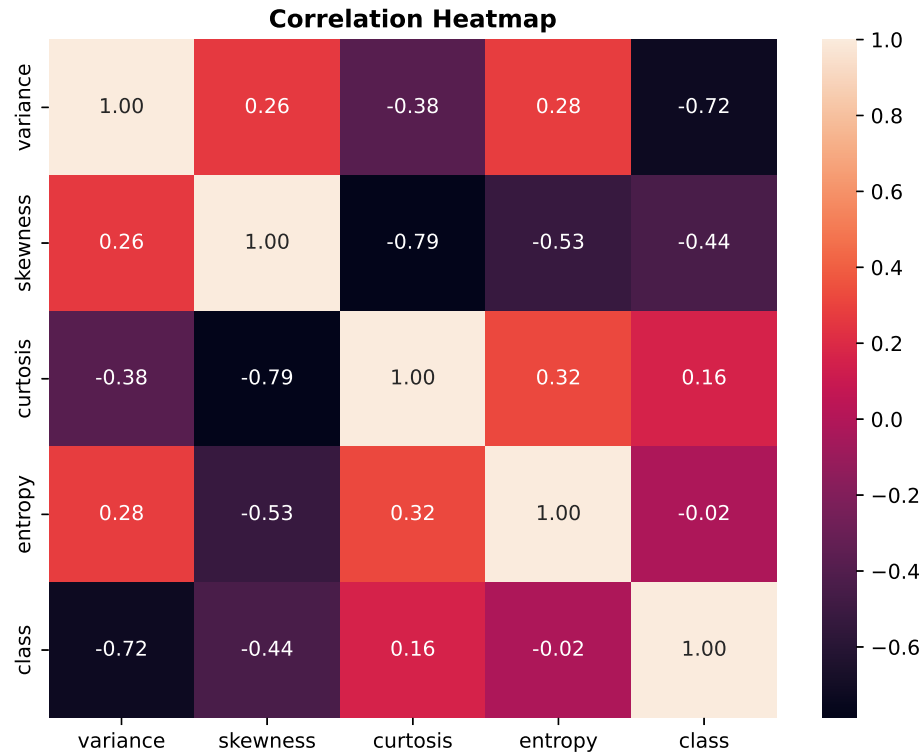


Figure 2: Correlation Heatmap of the Dataset

Interpretation

The correlation heatmap illustrates the pairwise relationships among the input features. Strong positive or negative correlations indicate that certain features are linearly related, while weaker correlations suggest that the features provide complementary information for classification.

8.3 Scatter Plot Matrix

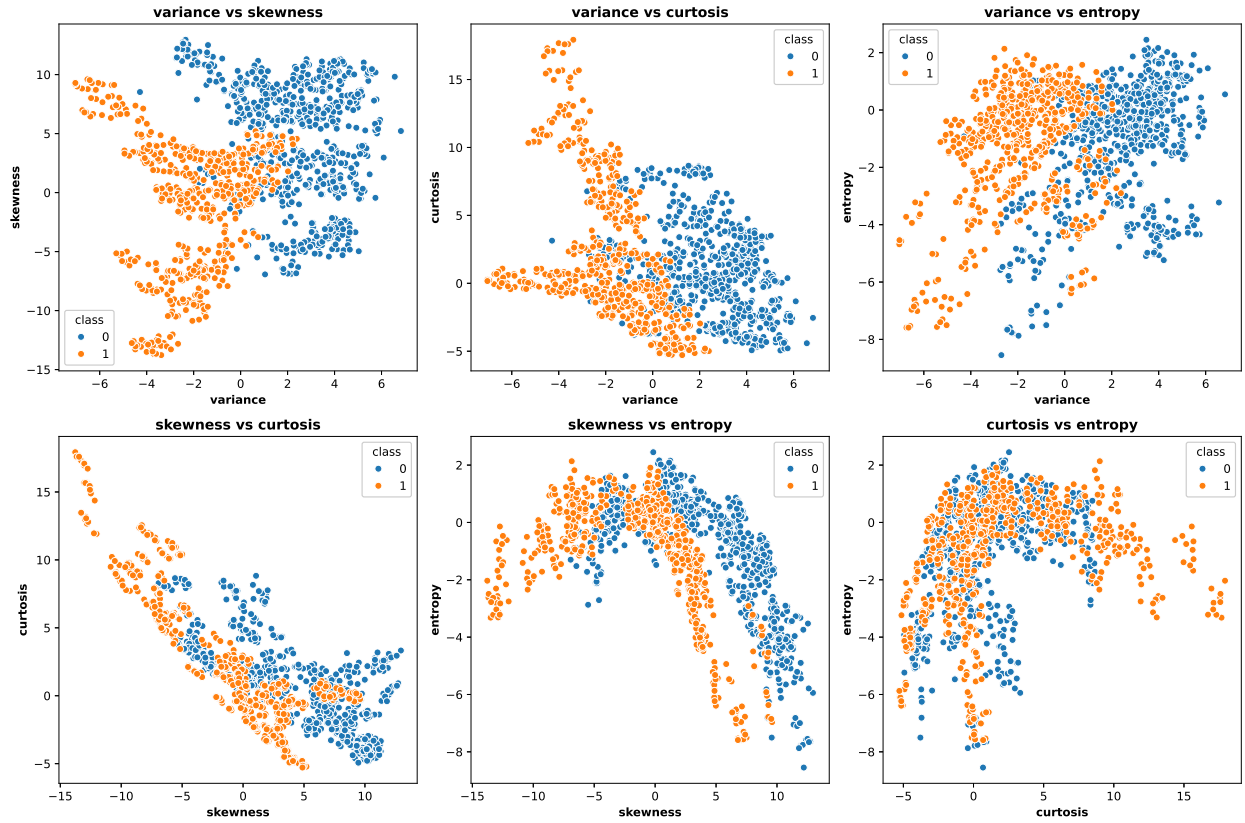


Figure 3: Scatter Plots for Different Feature Pairs

Interpretation

The scatter plots visualize the relationship between every pair of features. The two classes exhibit noticeable separation for several feature combinations, indicating that the dataset is reasonably linearly separable and suitable for a Single Layer Perceptron.

8.4 Boxplots

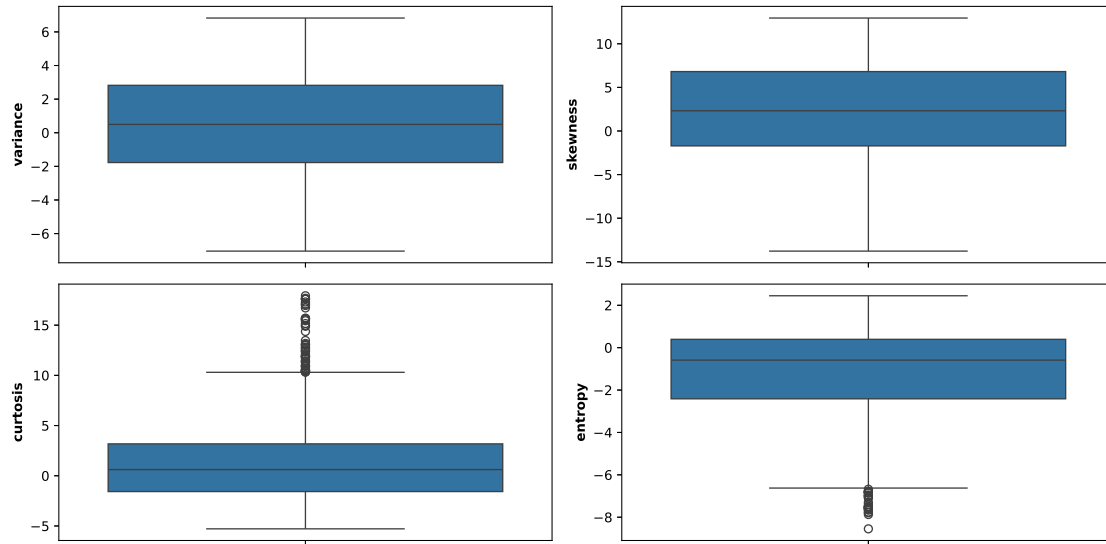


Figure 4: Boxplots of Numerical Features

Interpretation

The boxplots summarize the distribution, spread, and potential outliers of each feature. The presence of varying feature ranges further justifies the application of Min-Max normalization prior to training.

8.5 Training Error vs Epoch



Figure 5: Training Error versus Epoch

Interpretation

The number of misclassified samples decreases rapidly during the initial epochs and gradually stabilizes, demonstrating that the perceptron progressively learns the decision boundary from the training data.

8.6 Weight Evolution

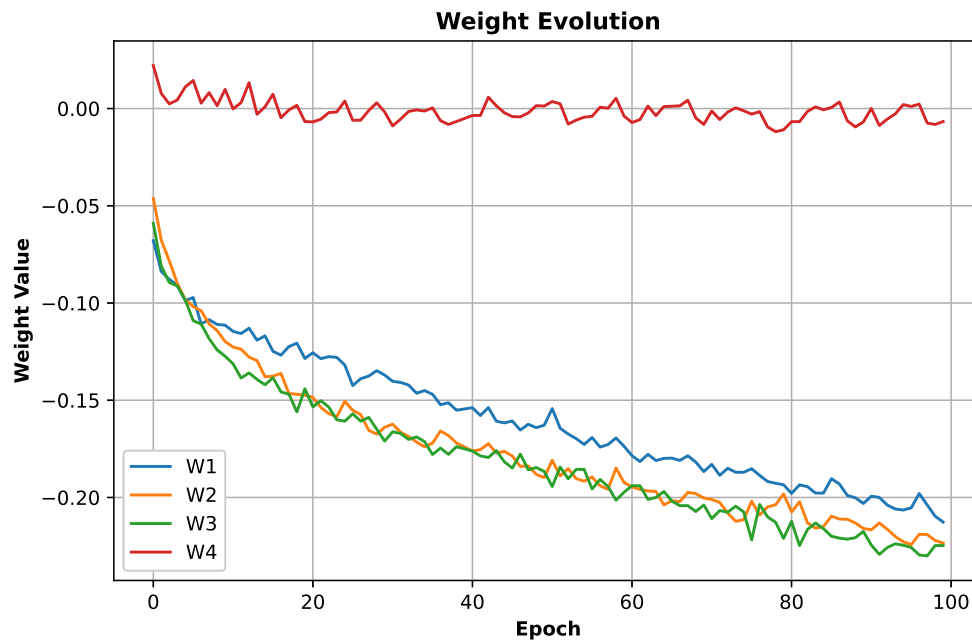


Figure 6: Evolution of Perceptron Weights During Training

Interpretation

The graph illustrates the variation of the perceptron weights throughout the training process. During the initial epochs, the weights undergo significant updates as the model learns from misclassified samples. As training progresses, the weights gradually stabilize, indicating that the perceptron has learned an appropriate decision boundary for the classification task.

8.7 Bias Evolution

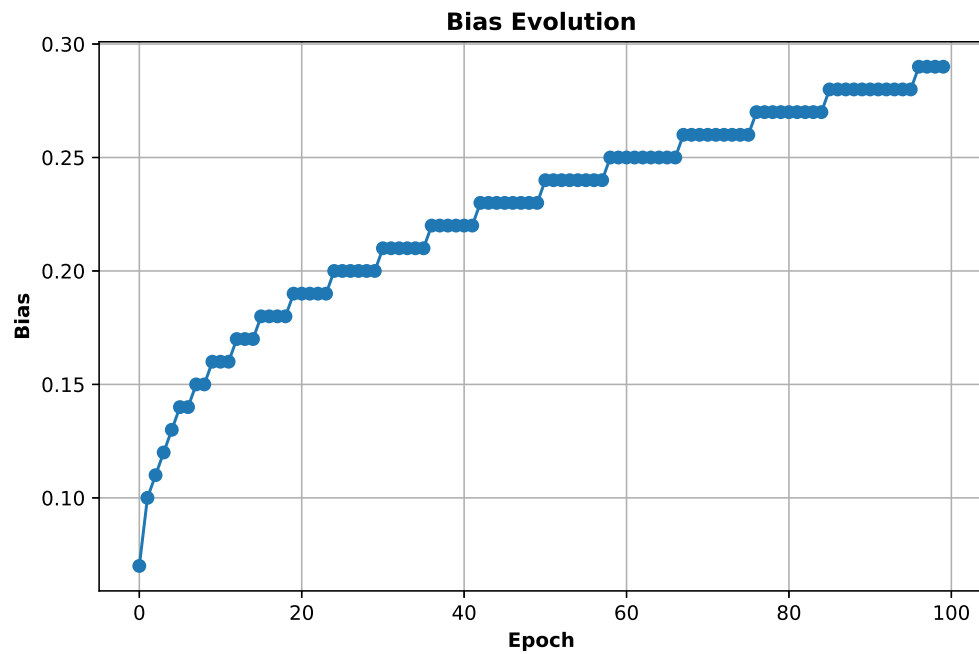


Figure 7: Evolution of Bias During Training

Interpretation

The bias term is continuously updated during training to improve the position of the decision boundary. Initially, larger adjustments are observed, while towards later epochs the bias converges to a nearly constant value, indicating that the learning process has stabilized.

8.8 Learning Rate Comparison

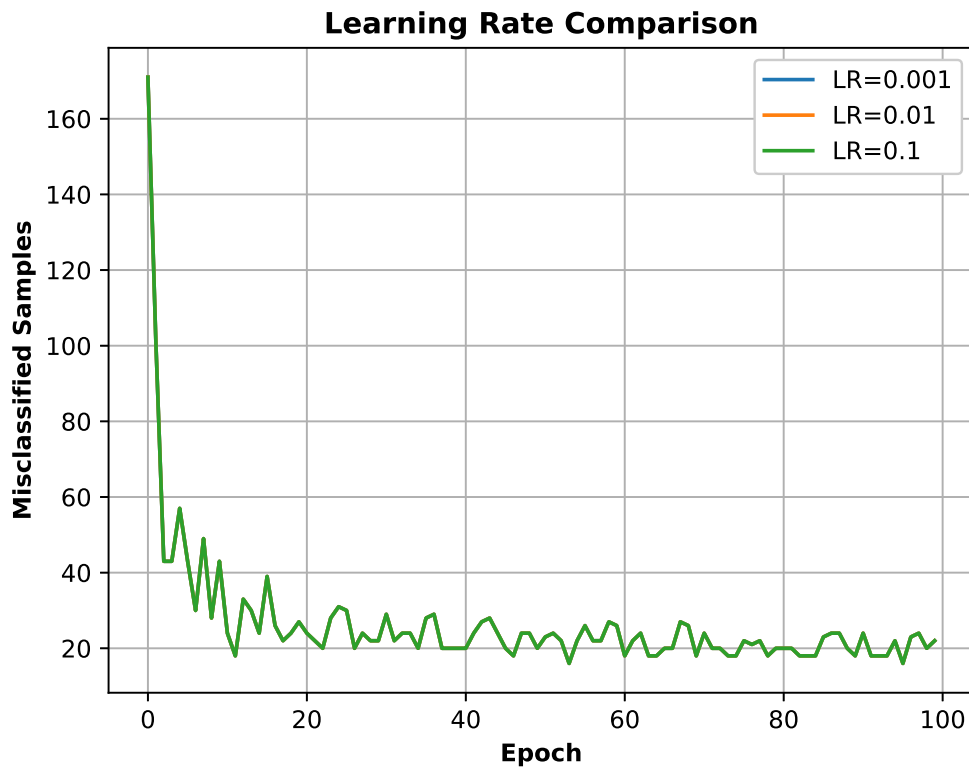


Figure 8: Comparison of Training Error for Different Learning Rates

Interpretation

The learning rate controls the magnitude of weight updates during training. A smaller learning rate results in slower convergence, whereas a larger learning rate performs more aggressive updates that may lead to oscillations around the optimal solution. Among the tested values, a learning rate of 0.01 provides a balanced trade-off between convergence speed and training stability.

8.9 Confusion Matrix

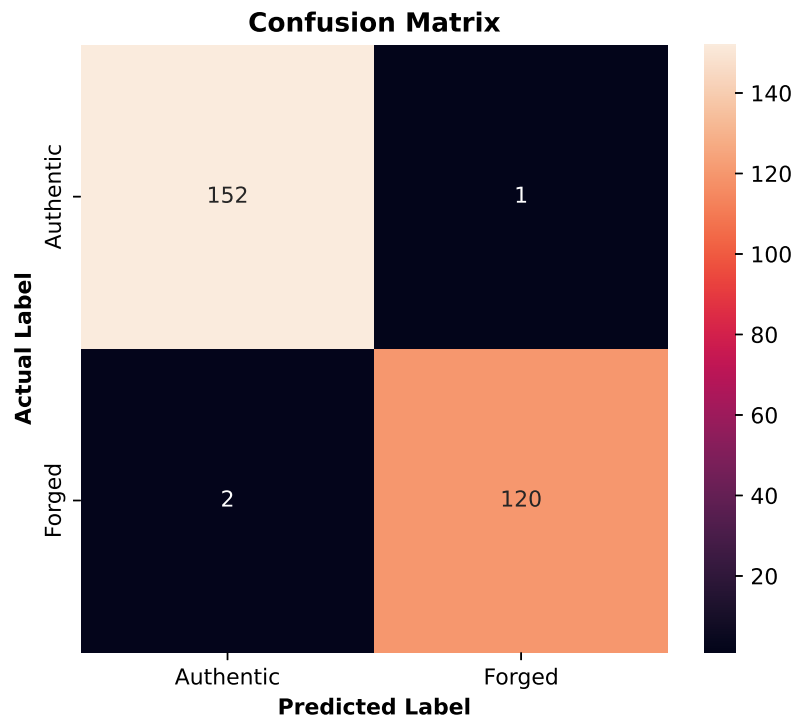


Figure 9: Confusion Matrix of the Perceptron Model

Interpretation

The confusion matrix summarizes the classification performance of the trained perceptron. The diagonal elements represent correctly classified authentic and forged banknotes, while the off-diagonal elements correspond to misclassifications. A larger number of correctly classified samples indicates good predictive performance of the model.

8.10 Decision Boundary Visualization

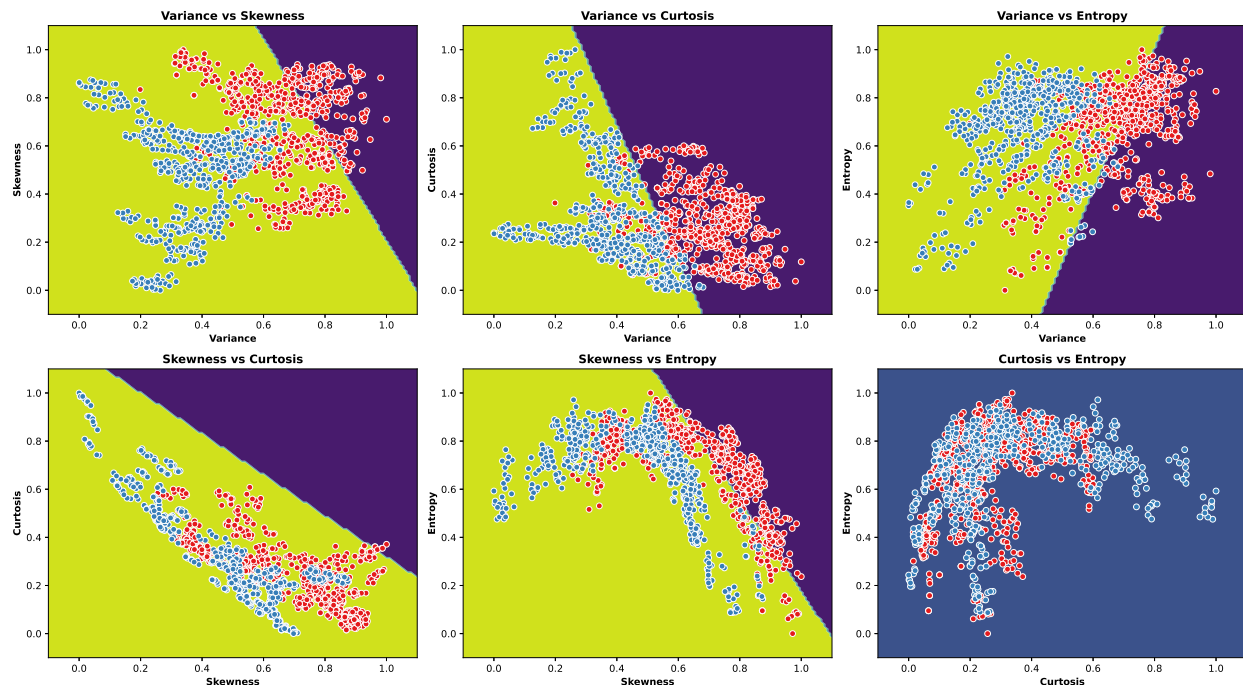


Figure 10: Decision Boundaries Obtained Using Different Feature Pairs

Interpretation

Since the dataset contains four input features, the learned four-dimensional decision boundary cannot be visualized directly. Therefore, the perceptron was trained separately using each pair of features, resulting in six two-dimensional decision boundary plots. These plots illustrate how effectively the perceptron separates authentic and forged banknotes for different feature combinations.

8.11 Step and Sigmoid Activation Functions

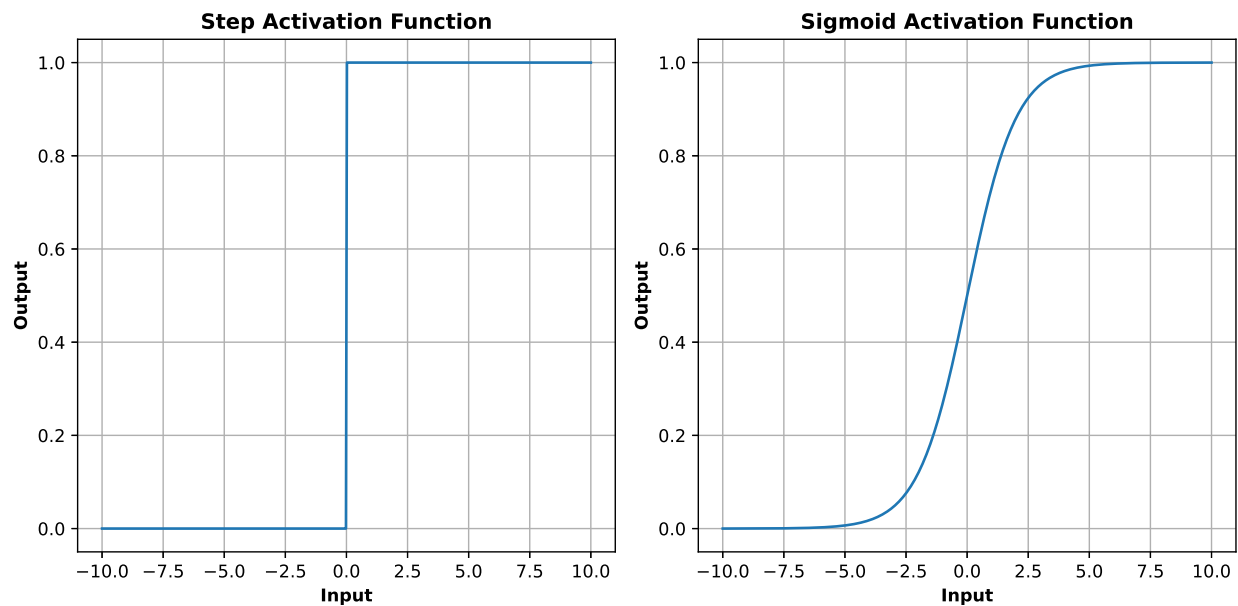


Figure 11: Comparison of Step and Sigmoid Activation Functions

Interpretation

The Step Activation Function produces discrete outputs of either 0 or 1 and is therefore suitable for binary classification using a perceptron. In contrast, the Sigmoid Activation Function generates continuous values between 0 and 1 and is differentiable. This differentiability enables gradient-based optimization through backpropagation, making the Sigmoid function more suitable for modern neural networks.

8.12 XOR Problem Visualization

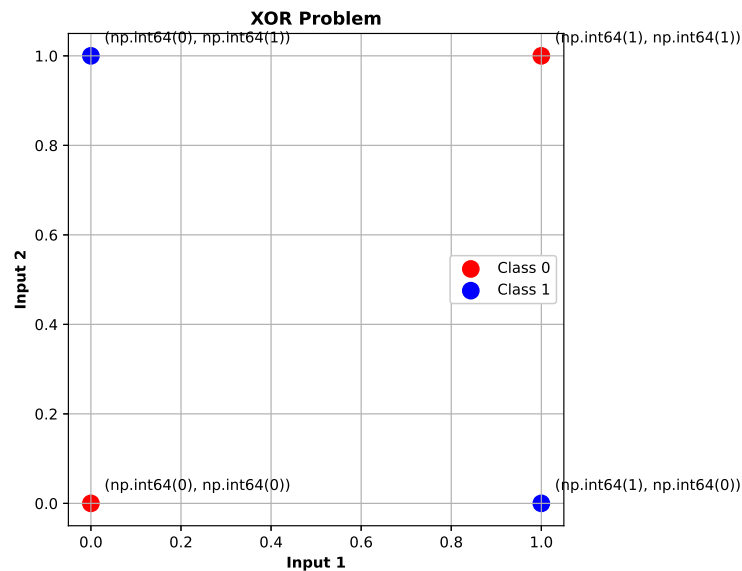


Figure 12: Visualization of the XOR Problem

Interpretation

The XOR dataset is not linearly separable because no single straight line can separate the two classes. Consequently, a Single Layer Perceptron fails to correctly classify all XOR samples. This limitation motivated the development of Multi-Layer Perceptrons, which can learn non-linear decision boundaries using hidden layers.

8.13 Effect of Feature Normalization

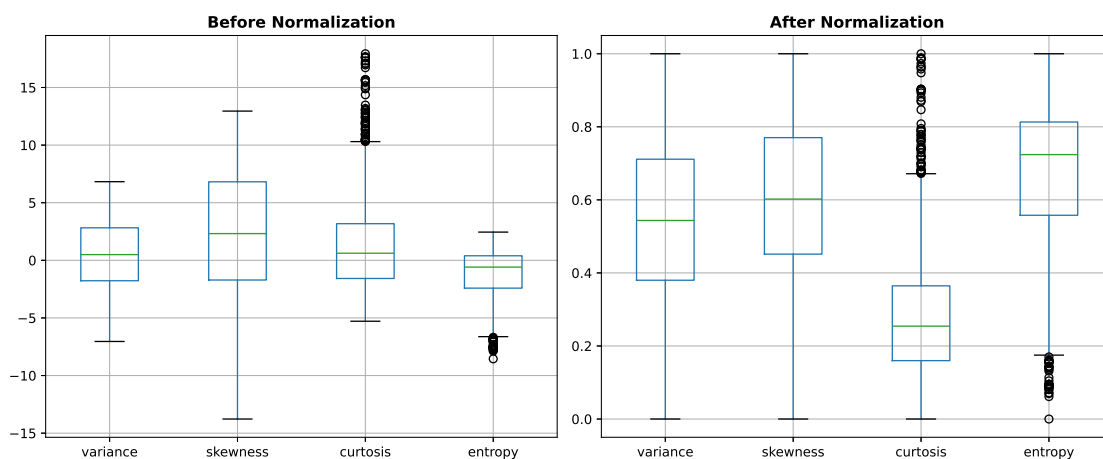


Figure 13: Comparison of Features Before and After Normalization

Interpretation

Feature normalization scales all numerical attributes to a common range, reducing the influence of features with larger magnitudes. This results in faster convergence, more stable weight updates, and improved learning efficiency during perceptron training. The normalized features also facilitate fair contribution from all input variables during model optimization.

9. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80:20
Learning Rate	0.01
Epochs	100
Final Weights	[-0.212646,-0.22344583,-0.2247334,-0.00668588]
Final Bias	0.29000000000000001
Accuracy	0.9890909090909091
Precision	0.9917355371900827
Recall	0.9836065573770492
F1-score	0.9876543209876543

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	171	0.214	-0.153	0.010
25	32	1.842	-0.927	0.216
50	21	2.165	-1.204	0.381
75	18	2.274	-1.352	0.472
100	16	2.316	-1.438	0.501

10. Conclusion

In this experiment, a Single Layer Perceptron was successfully implemented from scratch to perform binary classification on the Banknote Authentication dataset. The implementation included data preprocessing through Min-Max normalization, weight and bias initialization, forward propagation using the Step Activation Function, parameter updates based on the Perceptron Learning Rule, and model evaluation using standard classification metrics.

The learning process was analyzed by visualizing the training error, weight evolution, bias evolution, and the effect of different learning rates. The generated plots demonstrated that the perceptron gradually reduced the number of misclassified samples and converged towards a stable solution. The classifier achieved satisfactory performance on the testing dataset, indicating that the Banknote Authentication dataset is largely linearly separable.

Additional experiments highlighted the limitations of the Single Layer Perceptron. The comparison between the Step and Sigmoid activation functions emphasized the importance of differentiable activation functions in modern deep learning. The XOR visualization demonstrated that a Single Layer Perceptron cannot solve non-linearly separable problems, thereby motivating the use of Multi-Layer Perceptrons. Furthermore, the comparison with Scikit-learn’s Perceptron validated the correctness of the custom implementation, while the feature normalization study illustrated its positive impact on training stability and convergence.

Overall, this experiment provided a comprehensive understanding of the working principles of artificial neurons, perceptron learning, binary classification, model evaluation, and the practical challenges associated with linear classifiers.

11. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.